

Tecsup

Diseño y Desarrollo de Software

LABORATORIO N° 08

MVVM - ROOM

DOCENTE:

Juan José León Suiyon

CURSO:

Programación en Móviles
4to Ciclo

LABORATORIO — MVVM + Room

HuellitasVet: gestión de una veterinaria

Dato	Detalle
Docente	Juan León S.
Tecnología	Kotlin + Jetpack Compose + Material 3 (sin XML), Room 2.6.1, DataStore, Navigation Compose
Patrón	MVVM + Repository (interface)
Duración	1 semana (trabajo guiado en clase + trabajo autónomo)
Entrega	Repositorio GitHub (ramas main y mejora-ia) + README + PROMPTS.md + capturas
Nota	20 puntos (ver rúbrica)

1. El reto

Construirás HuellitasVet, la app de una veterinaria, con dos perfiles en el mismo dispositivo: el Dueño registra a sus mascotas, pide citas eligiendo servicio, fecha y hora, y consulta el historial clínico de cada mascota; el Veterinario ve la agenda, confirma citas, registra la atención (diagnóstico, tratamiento y peso), consulta a los pacientes y administra los servicios. Ambos perfiles comparten la misma base de datos Room: cuando el veterinario registra una atención, el dueño la ve en el historial de su mascota al instante.

> **Modo práctica (local)**
> Cada celular tiene su propia base de datos y ambos perfiles conviven en la app. Más adelante el Repository se conectará a un servicio web (Semana 9) y a la nube sin cambiar ViewModels ni pantallas; por eso es una interface. Usa solo datos ficticios (Ley 29733): nombres, correos y mascotas inventados.

Qué es nuevo en este laboratorio: una relación de dos niveles (dueño → mascotas → citas), una agenda con horarios que no se pueden repetir, reglas con fechas y horas (java.time), una transacción que actualiza dos tablas a la vez (la cita y el peso de la mascota), consultas con JOIN y SUM para el panel, y un avatar de mascota dibujado con Canvas.

Competencias: diseñar entidades y relaciones anidadas en Room, escribir DAOs con Flow y suspend, usar transacciones, aplicar MVVM con Repository, implementar reglas de negocio en el ViewModel, construir una UI Material 3 fiel a un diseño y trabajar con Git de forma profesional.

2. Cómo se trabaja: qué recibes y qué construyes

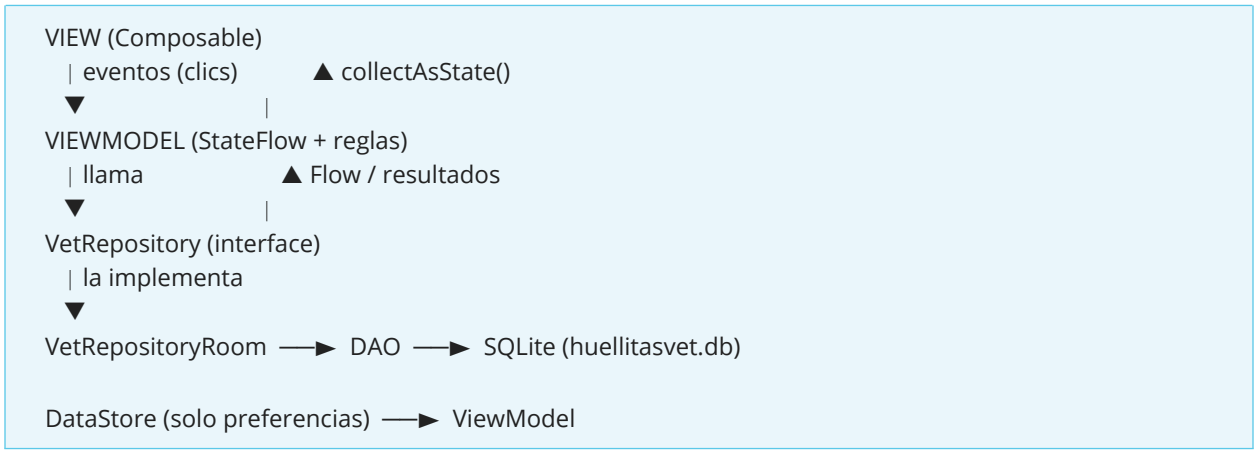
Recibirás un proyecto base que ya compila. Cada archivo del laboratorio tiene un nivel:

Nivel	Significado	Qué haces
ENTREGADO	Ya está hecho en el proyecto base	Léelo, úsalo y no lo modifiques (salvo que el

Nivel	Significado	Qué haces
		paso lo indique)
GUIADO	Archivo con esqueleto y comentarios // TODO	Completas lo que falta respetando las firmas dadas
TÚ LO HACES	El archivo no existe o está vacío	Lo creas desde cero siguiendo el contrato del paso

> **Ejemplo resuelto**
> El proyecto base incluye UN vertical completo de punta a punta: Servicio (entidad), ServicioDao, sus funciones en el Repository, ServiciosViewModel y la pantalla de servicios. Estúdialo primero: el resto de la app sigue exactamente el mismo patrón.

3. Arquitectura MVVM + Repository



Capa	Conoce a	NO debe
View (ui/)	Solo a su ViewModel	Usar DAO, Repository ni reglas de negocio
ViewModel	Solo al Repository (interface)	Conocer Composables ni el DAO
Repository	Los DAOs (en la implementación)	Conocer pantallas ni ViewModels
DAO / Room	SQL y entidades	Conocer ViewModels

Regla de oro: lectura que debe actualizar la pantalla → Flow (sin suspend). Escritura o acción única → suspend. No se reemplazan, se complementan.

4. Dependencias (ya incluidas en el proyecto base)

Revísalas en build.gradle.kts del módulo app. La versión de KSP debe coincidir con la de Kotlin; si Gradle marca error al sincronizar, revisa primero esto. El proyecto usa minSdk 26 para poder emplear java.time (LocalDate, LocalTime) sin librerías extra.

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose")
    id("com.google.devtools.ksp")        // versión = Kotlin (ej. 2.0.21-1.0.28)
}
dependencies {
    implementation(platform("androidx.compose:compose-bom:2024.09.00"))
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.material:material-icons-extended")
    implementation("androidx.activity:activity-compose:1.9.2")
    implementation("androidx.navigation:navigation-compose:2.8.2")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.6")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.6")
    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")
    ksp("androidx.room:room-compiler:2.6.1")
    implementation("androidx.datastore:datastore-preferences:1.1.1")
}
```

5. Base de datos: 4 tablas

Tabla	Columnas (tipo)	Relación
usuarios	id (Int, PK auto) · nombre · correo · clave · rol (String: CLIENTE | ADMIN)	—
servicios	id (PK auto) · nombre · descripcion · duracionMin (Int) · precio (Double) · activo (Boolean) · icono (String: heart, syringe, scissors)	—
mascotas	id (PK auto) · duenoId (FK→usuarios) · nombre · especie (String: PERRO | GATO | OTRO) · raza · anioNacimiento (Int) · peso (Double) · sexo (String: MACHO | HEMBRA)	N mascotas → 1 dueño
citas	id (PK auto) · mascotaId (FK→mascotas) · servicioId (FK→servicios) · fecha (String 'aaaa-MM-dd') · hora (String 'HH:mm') · nota · estado (String: PENDIENTE | CONFIRMADA | ATENDIDA | CANCELADA) · precio (Double) · diagnostico (String?) · tratamiento (String?) · pesoRegistrado (Double?) · fechaCreacion (Long)	N citas → 1 mascota; N citas → 1 servicio

> **¿Por qué fecha y hora son texto?**
> El texto 'aaaa-MM-dd' se ordena igual que las fechas (2026-10-21 va antes que 2026-10-22), se compara con = en SQL y se convierte con LocalDate.parse(). Así los alumnos se concentran en Room y no en convertidores de tipos.

> **¿Por qué guardar el precio en la cita?**
> Es una 'foto' del momento: si mañana la clínica cambia el precio de la vacuna, las citas antiguas conservan lo que costaron y los ingresos del panel no se alteran.

Datos iniciales (se cargan solo la primera vez): 4 servicios (Consulta general 30 min S/60, Vacunación 30 min S/45, Baño y corte 60 min S/50, Desparasitación 15 min S/25 inactivo), 2 dueños (ana@correo.test y luis@correo.test, clave 1234), 1 veterinario (vet@huellitas.test / admin), 4 mascotas (Luna, Max, Rocky, Michi) y unas 10 citas en distintos estados: algunas pasadas ATENDIDAS con diagnóstico (para el historial), algunas de hoy y otras de los próximos días.

6. Reglas de negocio

Estas reglas viven en los ViewModels (no en los Composables ni en los DAOs). Son lo que hace 'real' a la app:

Regla	Detalle
Visibilidad	El dueño solo ve SUS mascotas y las citas de sus mascotas. El veterinario ve todo
Registrar mascota	Nombre de al menos 2 caracteres, especie y sexo elegidos, año de nacimiento entre 1990 y el año actual, peso mayor que 0
Eliminar mascota	Solo si no tiene citas. Si tiene, se muestra un mensaje y no se borra
Pedir cita	Mascota del dueño, servicio ACTIVO, fecha desde hoy hasta 7 días, horarios de 09:00 a 17:30 cada 30 minutos. Si la fecha es hoy, solo horas futuras. Estado inicial PENDIENTE. Guarda el precio del servicio en ese momento
Horario único	La clínica atiende una cita por horario: si ya existe una cita NO cancelada con la misma fecha y hora, ese horario aparece deshabilitado (gris)
Cancelar (Dueño)	Solo citas PENDIENTE o CONFIRMADA y solo si faltan más de 2 horas para la cita
Estados (Veterinario)	PENDIENTE → CONFIRMADA. Desde PENDIENTE o CONFIRMADA puede CANCELAR. ATENDIDA solo se logra registrando la atención. ATENDIDA y CANCELADA son estados finales
Registrar atención	Diagnóstico de al menos 5 caracteres y peso mayor que 0. En UNA transacción: guarda diagnóstico, tratamiento y peso en la cita, la pasa a ATENDIDA y actualiza el peso de la mascota
Historial clínico	Citas ATENDIDAS de una mascota, la más reciente primero
Panel	Citas de hoy (sin contar canceladas), pendientes, atendidas, ingresos del mes (suma del precio de las citas ATENDIDAS desde el día 1) y alerta de citas por confirmar
Servicios	Un servicio con citas no se puede eliminar (solo desactivar). Un servicio inactivo no aparece al pedir cita

7. Estructura de paquetes y archivos

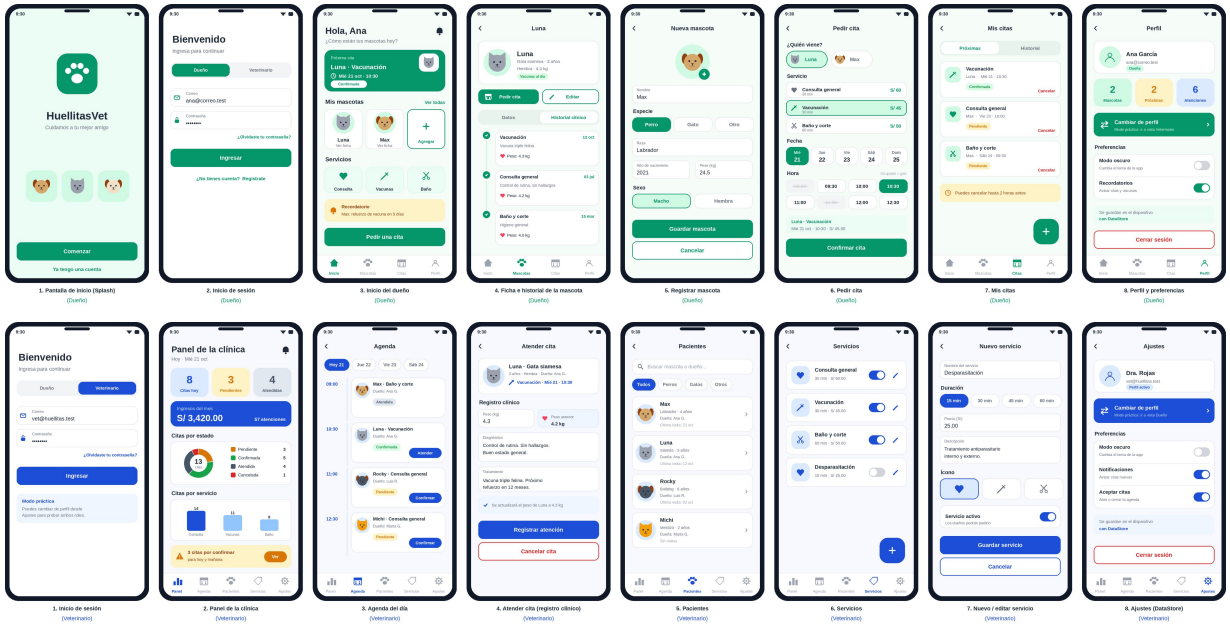
com.tecsup.huellitasvet — marcado con el nivel de cada archivo: [E] entregado, [G] guiado, [A] tú lo haces.

```
com.tecsup.huellitasvet
├── MainActivity.kt                [E]
├── VetApp.kt                      [G]  crea base y repository
│
├── data
│   ├── local
│   │   ├── entity
│   │   │   ├── Servicio.kt          [E]  ← ejemplo resuelto
│   │   │   ├── Usuario.kt           [A]
│   │   │   ├── Mascota.kt           [A]
│   │   │   └── Cita.kt              [A]
│   │   ├── dao
│   │   │   ├── ServicioDao.kt       [E]  ← ejemplo resuelto
│   │   │   ├── UsuarioDao.kt        [A]
│   │   │   ├── MascotaDao.kt        [A]
│   │   │   └── CitaDao.kt           [A]
│   │   ├── relation
│   │   │   ├── MascotaConDueno.kt    [A]
│   │   │   └── CitaConDetalle.kt     [A]
│   │   ├── VetDatabase.kt           [G]
│   │   └── DatosIniciales.kt        [G]
│   ├── repository
│   │   ├── VetRepository.kt         [E]  interface (el contrato)
│   │   └── VetRepositoryRoom.kt     [G]  Servicio listo; el resto TODO
│   └── prefs
│       └── PreferenciasStore.kt     [A]
│
├── model
│   ├── Rol.kt  EstadoCita.kt  Especie.kt  Sexo.kt   [E]
│   ├── ConteoServicio.kt  Horarios.kt            [E]
│   └── (Horarios genera los 18 horarios del día)
│
├── viewmodel
│   ├── VetViewModelFactory.kt       [E]
│   ├── ServiciosViewModel.kt        [G]  lista lista; guardar TODO
│   ├── SesionViewModel.kt           [A]
│   ├── InicioViewModel.kt           [A]
│   ├── MascotaViewModel.kt          [A]
│   ├── PedirCitaViewModel.kt        [A]
│   ├── MisCitasViewModel.kt         [A]
│   ├── PanelViewModel.kt            [A]
│   ├── AgendaViewModel.kt           [A]
│   ├── AtenderCitaViewModel.kt      [A]
│   └── PacientesViewModel.kt        [A]
│
└── ui
    ├── theme       Color.kt  Type.kt  Theme.kt            [E]
    ├── components   AppTopBar  VetButton  VetTextField
    │               EstadoCitaChip  MascotaAvatar  CitaCard
    │               FechaChip  HoraChip  HistorialItem      [E]
    │   └── graficos  GraficoDona  GraficoBarras    [E] (del lab. de gráficos)
    ├── navigation   Rutas.kt [E]   NavVet.kt [G]
```

```
├── dueno
│   ├── SplashScreen.kt          [E]
│   ├── LoginScreen.kt           [A]
│   ├── InicioScreen.kt          [A]
│   ├── MascotasScreen.kt        [A]
│   ├── FormularioMascotaScreen.kt  [A]
│   ├── FichaMascotaScreen.kt    [A]
│   ├── PedirCitaScreen.kt       [A]
│   ├── MisCitasScreen.kt        [A]
│   └── PerfilScreen.kt          [A]
└── veterinario
    ├── PanelScreen.kt           [A]
    ├── AgendaScreen.kt          [A]
    ├── AtenderCitaScreen.kt     [A]
    ├── PacientesScreen.kt       [A]
    ├── ServiciosScreen.kt       [E]  ← ejemplo resuelto
    ├── FormularioServicioScreen.kt [A]
    └── AjustesScreen.kt         [A]
```

8. Diseño: igual al mockup (Material 3)

Tu app debe verse como estas pantallas. Los componentes y el tema ya están entregados: úsalos, no pintes colores sueltos.



Uso	Dueño	Veterinario
Color primario	Esmeralda #059669	Azul #1D4ED8
Contenedor	#D1FAE5	#DBEAFE
Fondo	#F6FDF9	#F7F9FF
Estado de la cita	Pendiente ámbar #D97706 · Confirmada verde #16A34A · Atendida gris #475569 ·	

Uso	Dueño	Veterinario
	Cancelada rojo #DC2626	

- VetTheme(rol) entrega un ColorScheme según el perfil. En tus pantallas usa MaterialTheme.colorScheme.primary, nunca un color fijo.
- MascotaAvatar dibuja la carita de un perro, un gato u otro animal con Canvas (viene hecho). Úsalo en tarjetas, fichas y selectores.
- FechaChip y HoraChip son los selectores de la pantalla Pedir cita; HoraChip tiene un estado 'ocupado' (gris y tachado) y no se puede tocar.
- HistorialItem dibuja la línea de tiempo del historial clínico (fecha, servicio, diagnóstico y peso).
- Scaffold con NavigationBar: 4 ítems para el dueño (Inicio, Mascotas, Citas, Perfil) y 5 para el veterinario (Panel, Agenda, Pacientes, Servicios, Ajustes).
- Modo oscuro con darkColorScheme, activable desde Perfil o Ajustes y guardado en DataStore.

9. El ejemplo resuelto: estúdialo antes de empezar

Servicio.kt (entidad = tabla)

```
@Entity(tableName = "servicios")
data class Servicio(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String,
    val descripcion: String,
    val duracionMin: Int,
    val precio: Double,
    val activo: Boolean = true,
    val icono: String          // heart, syringe, scissors
)
```

ServicioDao.kt (lectura con Flow, escritura con suspend)

```
@Dao
interface ServicioDao {
    @Query("SELECT * FROM servicios ORDER BY nombre")
    fun observarTodos(): Flow<List<Servicio>>

    @Query("SELECT * FROM servicios WHERE activo = 1 ORDER BY nombre")
    fun observarActivos(): Flow<List<Servicio>>

    @Query("SELECT * FROM servicios WHERE id = :id")
    suspend fun obtener(id: Int): Servicio?

    @Query("SELECT COUNT(*) FROM servicios")
    suspend fun contar(): Int

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertar(s: Servicio): Long

    @Insert suspend fun insertarTodos(lista: List<Servicio>)
```

```
    @Update suspend fun actualizar(s: Servicio)
    @Delete suspend fun eliminar(s: Servicio)

    @Query("UPDATE servicios SET activo = :activo WHERE id = :id")
    suspend fun cambiarActivo(id: Int, activo: Boolean)
}
```

ServiciosViewModel.kt (Flow → StateFlow con stateIn)

```
class ServiciosViewModel(private val repo: VetRepository) : ViewModel() {
    val servicios: StateFlow<List<Servicio>> = repo.observarServicios()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    fun alternar(s: Servicio) {
        viewModelScope.launch { repo.cambiarActivo(s.id, !s.activo) }
    }
    // TODO (Paso 13): guardar(servicio) y eliminar(servicio)
}
```

ServiciosScreen.kt (la View solo observa y avisa)

```
@Composable
fun ServiciosScreen(vm: ServiciosViewModel, onEditar: (Int) -> Unit) {
    val servicios by vm.servicios.collectAsState()
    LazyColumn {
        items(servicios, key = { it.id }) { s ->
            ServicioItem(s, onToggle = { vm.alternar(s) }, onEditar = { onEditar(s.id) })
        }
    }
}
```

VetRepository.kt (el contrato: tú implementas cada función en VetRepositoryRoom)

```
interface VetRepository {
    // Usuarios
    suspend fun login(correo: String, clave: String): Usuario?

    // Servicios (ejemplo resuelto)
    fun observarServicios(): Flow<List<Servicio>>
    fun observarServiciosActivos(): Flow<List<Servicio>>
    suspend fun guardarServicio(s: Servicio)
    suspend fun eliminarServicio(s: Servicio)
    suspend fun cambiarActivo(id: Int, activo: Boolean)
    suspend fun servicioTieneCitas(id: Int): Boolean

    // Mascotas
    fun observarMascotasDeDueno(duenoId: Int): Flow<List<Mascota>>
    fun observarMascota(id: Int): Flow<Mascota?>
    fun observarTodasLasMascotas(): Flow<List<MascotaConDueno>>
    suspend fun guardarMascota(m: Mascota)
    suspend fun eliminarMascota(m: Mascota)
```

```
    suspend fun mascotaTieneCitas(id: Int): Boolean

    // Citas
    fun observarHorasOcupadas(fecha: String): Flow<List<String>>
    suspend fun crearCita(mascotaId: Int, servicioId: Int, fecha: String,
                          hora: String, nota: String): Long
    fun observarCitasDeDueno(duenoId: Int): Flow<List<CitaConDetalle>>
    fun observarCitasPorFecha(fecha: String): Flow<List<CitaConDetalle>>
    fun observarCita(id: Int): Flow<CitaConDetalle?>
    fun observarHistorial(mascotaId: Int): Flow<List<CitaConDetalle>>
    suspend fun cambiarEstadoCita(id: Int, estado: EstadoCita)
    suspend fun registrarAtencion(citaId: Int, diagnostico: String,
                                  tratamiento: String, peso: Double)

    // Panel
    fun observarCitasDeFecha(fecha: String): Flow<Int>
    fun observarCantidadPorEstado(estado: EstadoCita): Flow<Int>
    fun observarCitasPorServicio(): Flow<List<ConteoServicio>>
    fun observarIngresosDesde(fechaInicio: String): Flow<Double>
}
```

FASE 1 · SIN IA · Rama main

Construye HuellitasVet paso a paso

Reglas de la Fase 1

No uses IA (ni Gemini ni ChatGPT). Puedes usar la documentación oficial y tus apuntes. Haz un commit por paso, con el mensaje indicado. Si tu commit mezcla varios pasos, pierdes puntos de Git.

Paso 0 · Preparar el repositorio

```
# 1. Descarga el proyecto base y ábrelo en Android Studio
# 2. Sincroniza Gradle y ejecuta: debe abrir el Splash y la pantalla de Servicios
# 3. Crea tu repositorio vacío en GitHub y conéctalo
git init
git branch -M main
git remote add origin https://github.com/TU_USUARIO/HuellitasVet.git
git add .
git commit -m "chore: importa proyecto base HuellitasVet"
git push -u origin main
```

COMMIT 1 — **chore: importa proyecto base HuellitasVet**

La app base debe compilar y mostrar la lista de servicios.

Parte A · Datos (Room)

Paso 1 · Entidades Usuario, Mascota y Cita TÚ LO HACES

Objetivo: Crear las 3 entidades que faltan, siguiendo el modelo de Servicio.kt y las tablas de la sección 5.

Archivos:

- data/local/entity/Usuario.kt
- data/local/entity/Mascota.kt
- data/local/entity/Cita.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Usuario	@Entity(tableName = "usuarios"); PK autogenerada; rol como String guardando Rol.name
Mascota	@Entity(tableName = "mascotas"); duenoId con ForeignKey hacia Usuario; especie y sexo como String (enum.name); peso como Double
Cita	@Entity(tableName = "citas"); mascotaId con ForeignKey hacia Mascota y servicioId con ForeignKey hacia Servicio; fecha y hora como String; estado como String; diagnostico, tratamiento y pesoRegistrado pueden ser nulos; fechaCreacion = System.currentTimeMillis()
Índices	Cada columna FK debe tener su Index. Agrega además un índice en (fecha, hora) de Cita: acelera la consulta de horarios ocupados

> **Pista**
> Una clave foránea se declara en @Entity: foreignKeys = [ForeignKey(entity = Usuario::class, parentColumns = ["id"], childColumns = ["duenoId"])], indices = [Index("duenoId")]. Una entidad puede tener varias foreignKeys.

Verifica antes del commit: las entidades compilan (aún no están registradas en la base).

COMMIT 2 **feat(data): agrega entidades Usuario, Mascota y Cita**

Paso 2 · DAOs y relaciones anidadas TÚ LO HACES

Objetivo: Escribir las operaciones de acceso a datos y las relaciones que arman cada cita con su mascota, el dueño de la mascota y su servicio.

Archivos:

- data/local/dao/UsuarioDao.kt
- data/local/dao/MascotaDao.kt
- data/local/dao/CitaDao.kt
- data/local/relation/MascotaConDueno.kt
- data/local/relation/CitaConDetalle.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
UsuarioDao	login(correo, clave): Usuario? (suspend) · insertarTodos(lista) · contar(): Int (suspend)
MascotaDao	insertar, actualizar, eliminar (suspend) · actualizarPeso(id, peso) · observarPorDueno(duenoId), observarUna(id) y observarTodas(): Flow · contarCitas(mascotaId): Int (suspend)
CitaDao: escritura	insertar(c): Long · actualizarEstado(id, estado) · registrarAtencion(id, diagnostico, tratamiento, peso): pone estado ATENDIDA y guarda los tres datos · obtener(id): Cita? (suspend)
CitaDao: horarios	contarOcupadas(fecha, hora): Int (suspend, no cuenta CANCELADA) · observarHorasOcupadas(fecha): Flow<List<String>> con las horas de las citas no canceladas de ese día
CitaDao: lectura	observarPorDueno(duenoId) con INNER JOIN a mascotas · observarPorFecha(fecha) ordenada por hora · observarUna(id) · observarHistorial(mascotaId): solo ATENDIDA, más reciente primero. Todas devuelven Flow de CitaConDetalle con @Transaction
CitaDao: panel	contarPorFecha(fecha) (sin canceladas) · contarPorEstado(estado) · contarPorServicio(): Flow<List<ConteoServicio>> con JOIN a servicios y GROUP BY · ingresosDesde(fechaInicio): Flow<Double> con SUM(precio) de las ATENDIDAS (0.0 si no hay)
MascotaConDueno	@Embedded Mascota + @Relation al Usuario (dueño)
CitaConDetalle	@Embedded Cita + @Relation a MascotaConDueno (relación anidada, con entity = Mascota::class) + @Relation a Servicio

> **Pista**
> Citas de un dueño: SELECT c.* FROM citas c INNER JOIN mascotas m ON c.mascotaId = m.id WHERE m.duenoId = :duenoId ORDER BY c.fecha, c.hora. Una relación anidada se declara así: @Relation(entity = Mascota::class, parentColumn = "mascotaId", entityColumn = "id") val mascota: MascotaConDueno. Usa COALESCE(SUM(precio), 0.0) para evitar nulos.

Verifica antes del commit: el proyecto compila con KSP sin errores en los DAOs.

COMMIT 3 **feat(data): agrega DAOs y relaciones CitaConDetalle y MascotaConDueno**

Paso 3 · VetDatabase y datos iniciales GUIADO

Objetivo: Registrar las 4 entidades y los DAOs en la base de datos y sembrar los datos de prueba la primera vez.

Archivos:

- data/local/VetDatabase.kt
- data/local/DatosIniciales.kt
- VetApp.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Singleton	Una sola instancia de VetDatabase para toda la app
DatosIniciales	suspend fun sembrar(db): si usuarioDao().contar() == 0 inserta servicios, usuarios, mascotas y unas 10 citas con diagnósticos, como en la sección 5
Fechas de ejemplo	Calcula las fechas con LocalDate.now().plusDays(n).toString() para que siempre haya citas de hoy, pasadas y futuras. Las citas pasadas ATENDIDAS llevan diagnóstico, tratamiento y peso
VetApp	Application registrada en el AndroidManifest; al iniciar lanza sembrar(db) en una corrutina (Dispatchers.IO)

```
@Database(
    entities = [Servicio::class /* TODO: agrega las otras 3 */],
    version = 1,
    exportSchema = false
)
abstract class VetDatabase : RoomDatabase() {
    abstract fun servicioDao(): ServicioDao
    // TODO: usuarioDao(), mascotaDao() y citaDao()

    companion object {
        @Volatile private var INSTANCE: VetDatabase? = null
        fun obtener(context: Context): VetDatabase = INSTANCE ?: synchronized(this) {
            // TODO: Room.databaseBuilder(...) con nombre "huellitasvet.db" (singleton)
        }
    }
```

```
}
```

> **Pista**
> Si cambias una entidad después de ejecutar la app, sube el número de version o desinstala la app del emulador; si no, Room falla porque el esquema no coincide.

Verifica antes del commit: al ejecutar, la app abre sin errores. En Android Studio: View > Tool Windows > App Inspection > Database Inspector, y revisa que existan las 4 tablas con datos.

COMMIT 4 **feat(data): registra entidades y siembra datos iniciales en VetDatabase**

Paso 4 · VetRepositoryRoom completo GUIADO

Objetivo: Implementar en VetRepositoryRoom todas las funciones de la interface (hoy solo las de Servicio están hechas; el resto dice TODO).

Archivos:

- data/repository/VetRepositoryRoom.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
login	Llama al UsuarioDao y devuelve el Usuario o null
crearCita	Busca el servicio para copiar su precio, arma la Cita con estado PENDIENTE y fechaCreacion actual y la inserta; devuelve el id
registrarAtencion	Dentro de db.withTransaction { ... }: obtiene la cita, llama a citaDao.registrarAtencion y actualiza el peso de la mascota con mascotaDao.actualizarPeso. Si algo falla, no debe quedar nada a medias
servicioTieneCitas y mascotaTieneCitas	Devuelven true si el conteo del DAO es mayor que 0
observar*	Solo delegan en los DAOs; no deben usar suspend
cambiarEstadoCita	Guarda estado.name en la tabla

> **Pista**
> db.withTransaction es una función suspend de room-ktx: import androidx.room.withTransaction. Piensa: ¿esta función cambia datos una sola vez o debe avisar cuando cambian? La primera es suspend y la segunda es Flow.

Verifica antes del commit: la clase compila sin ningún TODO() pendiente.

COMMIT 5 **feat(data): implementa VetRepositoryRoom con todas las operaciones**

Paso 5 · Preferencias con DataStore TÚ LO HACES

Objetivo: Guardar en el dispositivo el perfil activo, el modo oscuro, las notificaciones y si la clínica acepta citas.

Archivos:

- data/prefs/PreferenciasStore.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Claves	perfilActivo (String: CLIENTE | ADMIN | vacío), usuarioId (Int), modoOscuro (Boolean), notificaciones (Boolean, por defecto true), aceptarCitas (Boolean, por defecto true)
Lectura	Un Flow por cada preferencia
Escritura	Funciones suspend: guardarSesion(rol, usuarioId), guardarModoOscuro(b), guardarNotificaciones(b), guardarAceptarCitas(b), cerrarSesion()
Creación	Extensión val Context.dataStore by preferencesDataStore("ajustes") declarada una sola vez, a nivel de archivo

> **Pista**
> DataStore es para preferencias pequeñas (clave-valor). Las mascotas, citas y servicios NO van en DataStore: van en Room.

Verifica antes del commit: compila; la usarás en el Paso 6.

COMMIT 6 **feat(data): agrega PreferenciasStore con DataStore**

Parte B · Flujo del Dueño

Paso 6 · SesionViewModel y LoginScreen TÚ LO HACES

Objetivo: Permitir el inicio de sesión con selector de rol y recordar el perfil activo.

Archivos:

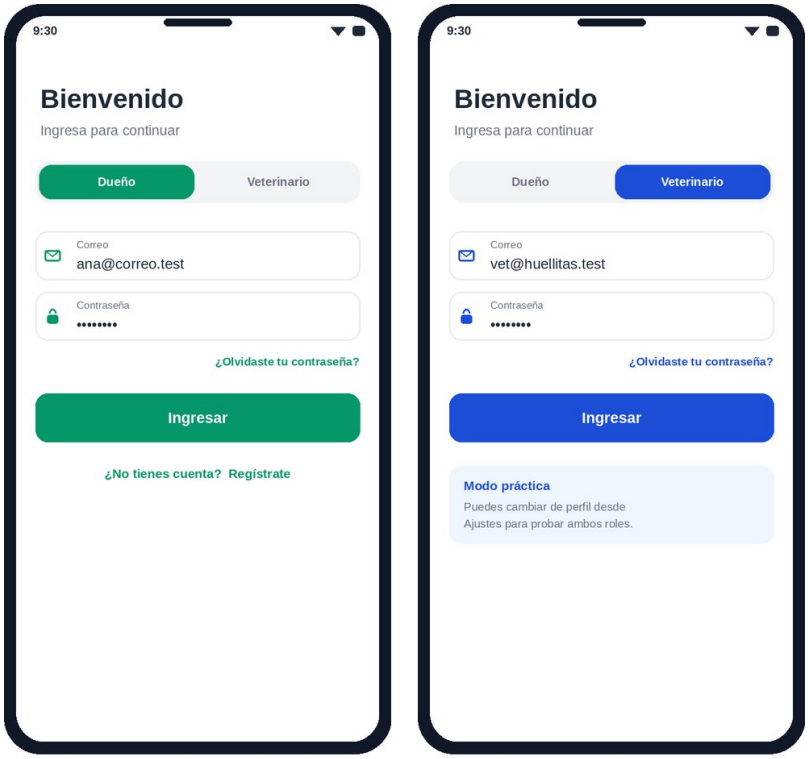
- viewmodel/SesionViewModel.kt
- ui/dueno/LoginScreen.kt
- ui/navigation/NavVet.kt (agregar el destino)

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
SesionViewModel	Expone usuarioActual: StateFlow<Usuario?>, error: StateFlow<String?> y modoOscuro. Función login(correo, clave, rolSeleccionado)
Validaciones	Campos vacíos → mensaje de error. Credenciales incorrectas → 'Correo o contraseña incorrectos'. Si el rol del usuario no coincide con la pestaña elegida → mensaje de error
Éxito	Guarda el perfil en DataStore y navega a Inicio (dueño) o Panel

Elemento	Debe cumplir
	(veterinario)
LoginScreen	Un solo Composable usado por ambos roles: selector Dueño / Veterinario, 2 campos (VetTextField) y botón 'Ingresar' del color del rol
Splash	Si ya hay perfil guardado, salta el login y va directo a su pantalla principal

Resultado esperado:



2. Inicio de sesión (Dueño)

1. Inicio de sesión (Veterinario)

Verifica antes del commit: entras como ana@correo.test / 1234 y como vet@huellitas.test / admin; con datos incorrectos aparece el error.

COMMIT 7 **feat(viewmodel): agrega SesionViewModel y login con selector de rol**

Paso 7 · Inicio y lista de mascotas TÚ LO HACES

Objetivo: Mostrar al dueño su próxima cita, sus mascotas y accesos rápidos.

Archivos:

- viewmodel/InicioViewModel.kt
- ui/dueno/InicioScreen.kt
- ui/dueno/MascotasScreen.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
InicioViewModel	Recibe el dueño actual y expone mascotas: StateFlow<List<Mascota>> y proximaCita: StateFlow<CitaConDetalle?>. La próxima cita es la primera cita PENDIENTE o CONFIRMADA con fecha y hora desde ahora, ordenadas por fecha y hora (se calcula en el ViewModel a partir de observarCitasDeDueno)
InicioScreen	Saludo con el nombre, tarjeta verde de próxima cita (o un mensaje 'Aún no tienes citas'), carrusel de mascotas con MascotaAvatar, botón 'Agregar' y botón 'Pedir una cita'
MascotasScreen	Lista de todas las mascotas del dueño con avatar, especie, raza y edad (año actual − anioNacimiento) y FAB para agregar
Navegación	Tocar una mascota abre su ficha (Paso 8); 'Agregar' abre el formulario (Paso 8); 'Pedir una cita' abre el Paso 9

> **Pista**
> Para comparar fechas y horas usa LocalDateTime.of(LocalDate.parse(c.cita.fecha), LocalTime.parse(c.cita.hora)) y compáralo con LocalDateTime.now().

Resultado esperado:



3. Inicio del dueño (Dueño)

Verifica antes del commit: ana@correo.test ve a Luna y Max, y la tarjeta muestra su próxima cita real.

COMMIT 8 **feat(ui): agrega inicio del dueno y lista de mascotas**

Paso 8 · Registrar mascota y ficha con historial TÚ LO HACES

Objetivo: Permitir registrar y editar mascotas, y mostrar su ficha con el historial clínico.

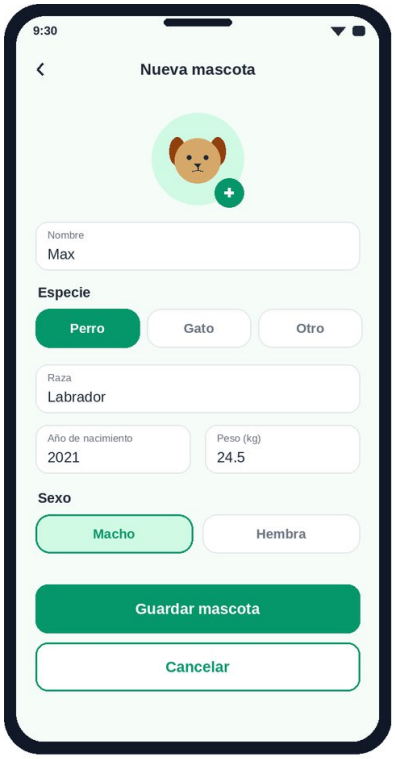
Archivos:

- viewmodel/MascotaViewModel.kt
- ui/dueno/FormularioMascotaScreen.kt
- ui/dueno/FichaMascotaScreen.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
MascotaViewModel	guardar(mascota) con validaciones, eliminar(mascota) y historial(mascotaId): StateFlow<List<CitaConDetalle>>. Los errores se exponen por campo
Validaciones	Nombre ≥ 2 caracteres, especie y sexo elegidos, año entre 1990 y el año actual, peso > 0. Los errores salen junto al campo
Eliminar	Si la mascota tiene citas (mascotaTieneCitas), no se elimina y se muestra un mensaje
FormularioMascotaScreen	Avatar según la especie elegida, campos nombre, raza, año y peso, chips de especie (Perro, Gato, Otro) y de sexo. Sirve para crear y para editar (recibe un id opcional)
FichaMascotaScreen	Cabecera con avatar, nombre, especie, raza, edad y peso actual; botones 'Pedir cita' y 'Editar'; pestañas Datos / Historial clínico. El historial usa HistorialItem y observa observarHistorial
Historial vacío	Si no hay atenciones, muestra 'Aún no tiene atenciones'

Resultado esperado:



5. Registrar mascota
(Dueño)



4. Ficha e historial de la mascota
(Dueño)

Verifica antes del commit: registras a una mascota nueva y aparece en Inicio; en la ficha de Luna ves 3 atenciones, la más reciente arriba.

COMMIT 9 **feat(ui): agrega formulario de mascota y ficha con historial clinico**

Paso 9 · Pedir cita con horarios TÚ LO HACES

Objetivo: Permitir al dueño elegir mascota, servicio, fecha y hora, sin chocar con otras citas.

Archivos:

- viewmodel/PedirCitaViewModel.kt
- ui/dueno/PedirCitaScreen.kt

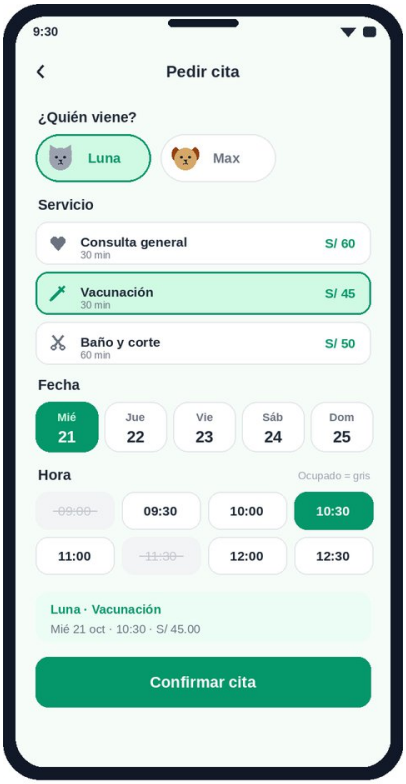
Contrato (lo que debe cumplir):

Elemento	Debe cumplir
PedirCitaViewModel	Estados: mascota, servicio, fecha y hora elegidos; horasOcupadas: StateFlow que cambia con la fecha (usa flatMapLatest sobre observarHorasOcupadas). Función confirmar() que valida y llama a repo.crearCita
Fechas	Los chips muestran de hoy a 7 días (día de la semana y número). Por defecto: hoy
Horarios	Se generan los 18 horarios de 09:00 a 17:30 (clase Horarios). Un horario ocupado aparece gris y tachado y no se puede elegir. Si la fecha es hoy, los horarios que ya pasaron también se deshabilitan

Elemento	Debe cumplir
Servicios	Se cargan con observarServiciosActivos(): los servicios inactivos NO aparecen. Muestra duración y precio
Resumen y botón	Una tarjeta resume mascota, servicio, fecha, hora y precio; el botón 'Confirmar cita' se habilita solo cuando todo está elegido
Al confirmar	Vuelve a Mis citas con la nueva cita como PENDIENTE. Si el horario se ocupó mientras tanto, muestra un mensaje y no crea la cita

> **Pista**
> Antes de crear, vuelve a consultar si el horario sigue libre: dos citas no pueden compartir fecha y hora. Si la fecha cambia, limpia la hora elegida.

Resultado esperado:



6. Pedir cita
(Dueño)

Verifica antes del commit: pides una cita y esa hora aparece gris al intentar otra; si eliges la fecha de hoy, no puedes elegir una hora pasada.

COMMIT 10 **feat(ui): agrega pedir cita con horarios disponibles**

Paso 10 · Mis citas y cancelación TÚ LO HACES

Objetivo: Listar las citas del dueño en Próximas e Historial y permitir cancelar con la regla de las 2 horas.

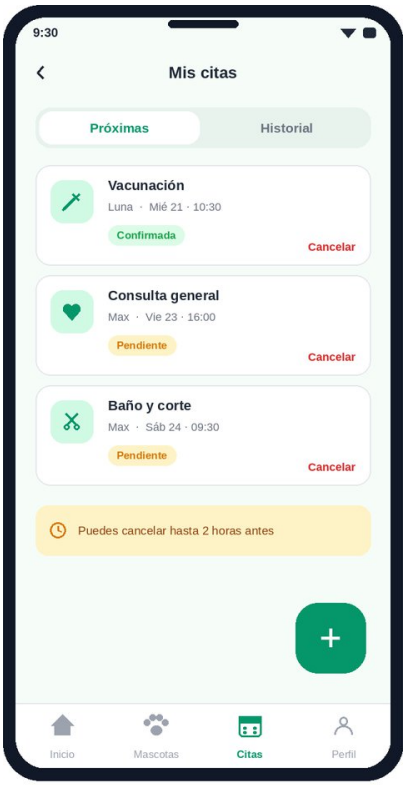
Archivos:

- viewmodel/MisCitasViewModel.kt
- ui/dueno/MisCitasScreen.kt
- ui/dueno/PerfilScreen.kt (se completa en el Paso 13)

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
MisCitasViewModel	proximas e historial: StateFlow. Próximas = PENDIENTE y CONFIRMADA desde ahora; Historial = ATENDIDA y CANCELADA y las pasadas. Función cancelar(cita)
Regla de cancelación	Solo PENDIENTE o CONFIRMADA y solo si faltan más de 2 horas (Duration.between(ahora, fechaHoraCita)). Si no se cumple, el botón no aparece o muestra el mensaje 'Ya no puedes cancelar esta cita'
MisCitasScreen	Pestañas Próximas / Historial, CitaCard con EstadoCitaChip, botón 'Cancelar' donde corresponde, aviso 'Puedes cancelar hasta 2 horas antes' y FAB para pedir una cita
Confirmación	Cancelar pide confirmación con un AlertDialog
Tiempo real	Si el veterinario confirma o atiende una cita, la lista cambia sola

Resultado esperado:



7. Mis citas
(Dueño)

Verifica antes del commit: cancelas una cita lejana; intentas cancelar una de dentro de 1 hora y no te deja.

COMMIT 11 **feat(ui): agrega mis citas con cancelacion y regla de 2 horas**

Parte C · Flujo del Veterinario

Paso 11 · Panel de la clínica y agenda TÚ LO HACES

Objetivo: Crear el panel con contadores, ingresos y gráficos (reutilizados del laboratorio de Canvas) y la agenda diaria.

Archivos:

- viewmodel/PanelViewModel.kt
- viewmodel/AgendaViewModel.kt
- ui/veterinario/PanelScreen.kt
- ui/veterinario/AgendaScreen.kt

Contrato (lo que debe cumplir):

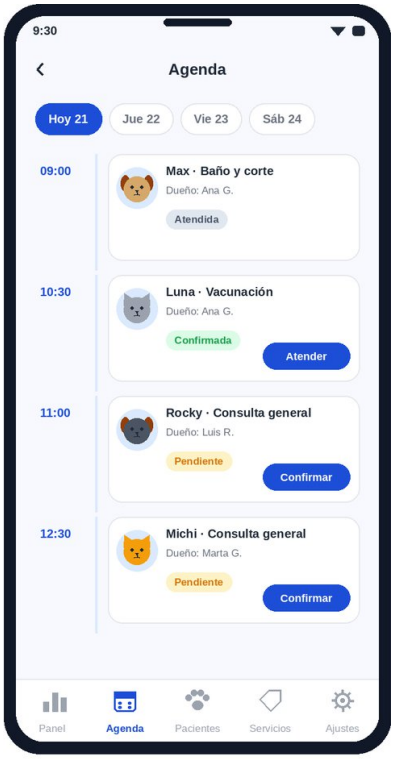
Elemento	Debe cumplir
PanelViewModel	citasHoy, pendientes, atendidas: StateFlow<Int>; ingresosMes: StateFlow<Double> (desde el día 1 del mes); porEstado y porServicio: StateFlow<List<DatoGrafico>>
PanelScreen	3 contadores de color, tarjeta azul de ingresos del mes, GraficoDona (citas por estado), GraficoBarras (citas por servicio) y tarjeta de alerta con botón 'Ver' si hay citas pendientes
AgendaViewModel	fechaSeleccionada (por defecto hoy) y citas: StateFlow del día elegido, ordenadas por hora. Funciones confirmar(cita) y cancelar(cita) que respetan las reglas de estado
AgendaScreen	Chips de los próximos días, línea de tiempo por hora con tarjetas (avatar, servicio, dueño y estado) y botón 'Confirmar' (si PENDIENTE) o 'Atender' (si CONFIRMADA)

> **Pista**
> GraficoDona y GraficoBarras ya vienen en ui/components/graficos y reciben List<DatoGrafico>. Tu trabajo es convertir los datos de Room a ese formato en el ViewModel. Para el día 1 del mes: LocalDate.now().withDayOfMonth(1).toString().

Resultado esperado:



2. Panel de la clínica
(Veterinario)



3. Agenda del día
(Veterinario)

Verifica antes del commit: los contadores cambian cuando, como dueño, pides una cita; al confirmar una cita desde la agenda, el dueño la ve CONFIRMADA.

COMMIT 12 **feat(veterinario): agrega panel con graficos y agenda del dia**

Paso 12 · Atender cita y pacientes TÚ LO HACES

Objetivo: Registrar la atención clínica de una cita y consultar a los pacientes.

Archivos:

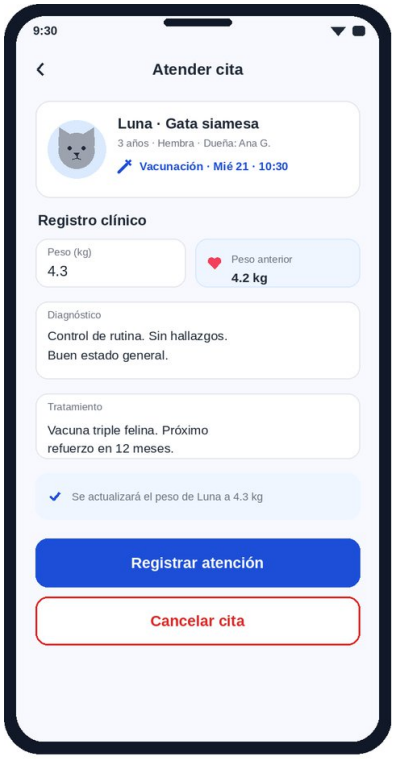
- viewmodel/AtenderCitaViewModel.kt
- ui/veterinario/AtenderCitaScreen.kt
- viewmodel/PacientesViewModel.kt
- ui/veterinario/PacientesScreen.kt

Contrato (lo que debe cumplir):

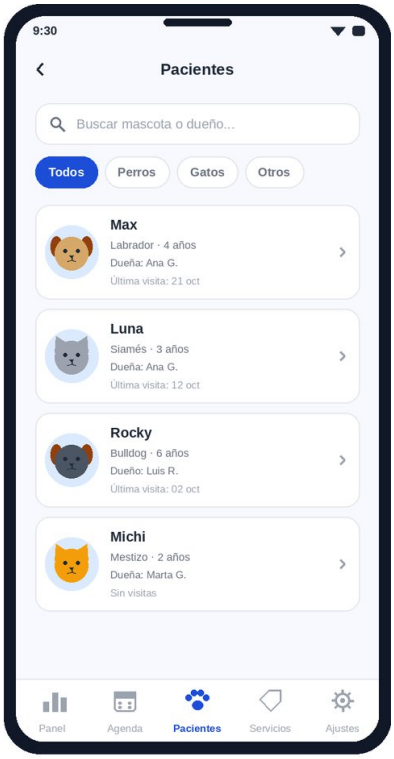
Elemento	Debe cumplir
AtenderCitaViewModel	cita: StateFlow<CitaConDetalle?>; registrar(diagnostico, tratamiento, peso) con validaciones y cancelar()
Validaciones	Diagnóstico ≥ 5 caracteres y peso > 0. Solo se puede atender una cita PENDIENTE o CONFIRMADA
Transacción	registrar llama a repo.registrarAtencion: la cita queda ATENDIDA y el peso de la mascota se actualiza en la misma operación

Elemento	Debe cumplir
AtenderCitaScreen	Cabecera con avatar, mascota, dueño, servicio y hora; campo de peso con el peso anterior; diagnóstico y tratamiento; aviso 'Se actualizará el peso de...'; botones 'Registrar atención' y 'Cancelar cita'
PacientesViewModel	Lista de MascotaConDueno filtrada por texto (nombre de mascota o de dueño) y por especie con combine
PacientesScreen	Buscador, chips Todos / Perros / Gatos / Otros y tarjetas con avatar, raza, edad, dueño y última visita. Tocar una tarjeta abre la ficha con historial

Resultado esperado:



4. Atender cita (registro clínico)
(Veterinario)



5. Pacientes
(Veterinario)

Verifica antes del commit: atiendes una cita, el dueño ve la atención en el historial de su mascota y el peso de la ficha cambió.

COMMIT 13 **feat(veterinario): agrega atencion de citas y pacientes**

Paso 13 · Servicios, perfil, ajustes y cambio de perfil TÚ LO HACES

Objetivo: Completar la gestión de servicios, el perfil del dueño y los ajustes del veterinario, y cerrar el ciclo con el cambio de perfil.

Archivos:

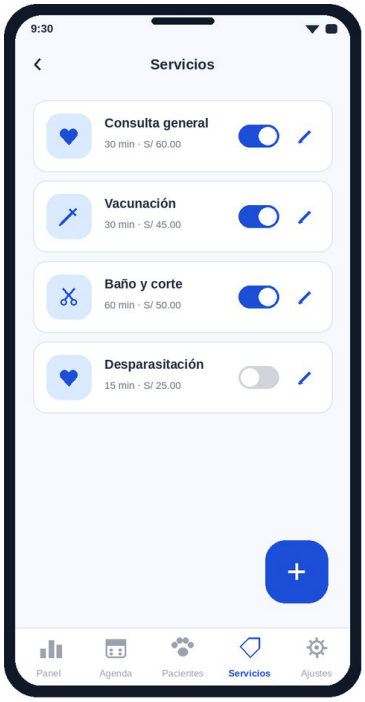
- viewmodel/ServiciosViewModel.kt (completar TODO)
- ui/veterinario/FormularioServicioScreen.kt

- ui/dueno/PerfilScreen.kt
- ui/veterinario/AjustesScreen.kt
- ui/navigation/NavVet.kt (completar rutas y grafo de cada perfil)

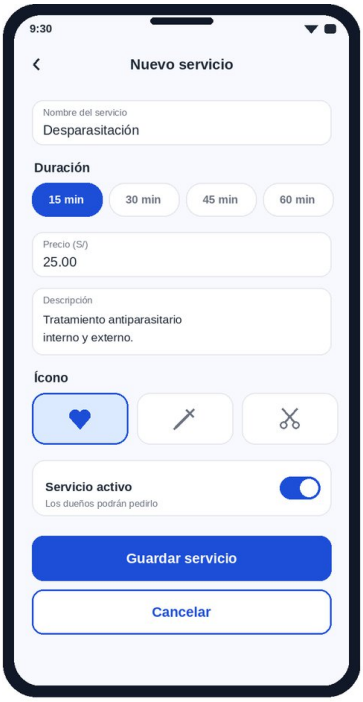
Contrato (lo que debe cumplir):

Elemento	Debe cumplir
ServiciosViewModel	Completa guardar(servicio) (nuevo o editado) y eliminar(servicio). Un servicio con citas no se elimina (muestra un mensaje); solo se desactiva
FormularioServicioScreen	Nombre (obligatorio), duración (15, 30, 45 o 60 min), precio > 0, descripción, ícono y switch 'Servicio activo'. Sirve para crear y para editar
PerfilScreen (dueño)	Datos del usuario y contadores de mascotas, citas próximas y atenciones; switches de modo oscuro y recordatorios; botón 'Cambiar de perfil' y 'Cerrar sesión'
AjustesScreen (veterinario)	Datos del usuario, 'Cambiar de perfil', switches de modo oscuro, notificaciones y 'Aceptar citas', y 'Cerrar sesión'
Aceptar citas	Si está apagado, el dueño ve el aviso 'La clínica no está recibiendo citas' y no puede pedir una
Cambiar de perfil	Guarda el otro perfil y navega a su grafo; limpia la pila de navegación (popUpTo)
Persistencia	Al cerrar y abrir la app, el modo oscuro y el perfil se conservan

Resultado esperado:



6. Servicios
(Veterinario)



7. Nuevo / editar servicio
(Veterinario)



8. Ajustes (DataStore)
(Veterinario)

Verifica antes del commit: desactivas un servicio y deja de aparecer al pedir cita; activas modo oscuro, cierras la app por completo y al abrir sigue oscuro.

COMMIT 14 **feat: agrega servicios, perfil, ajustes y cambio de perfil**

Paso 14 · Pruebas de aceptación y README

Marca cada prueba en el emulador. Todas deben pasar antes de cerrar la Fase 1.

Prueba	Resultado esperado	OK
Login con clave incorrecta	Mensaje de error, no entra	☐
Dueño: registrar mascota sin nombre	Errores junto a cada campo, no se guarda	☐
Dueño: eliminar una mascota con citas	Mensaje de aviso y no se elimina	☐
Dueño: pedir cita	Aparece en Mis citas como PENDIENTE y con el precio del servicio	☐
Dueño: elegir un horario ocupado	El horario aparece gris y no se puede elegir	☐
Veterinario: agenda	La cita aparece en su hora y puede confirmarla; el dueño la ve CONFIRMADA	☐
Veterinario: atender una cita	Pasa a ATENDIDA, el dueño la ve en el historial y el peso de la mascota se actualizó	☐
Dueño: cancelar una cita a menos de 2 horas	No se permite cancelar	☐
Veterinario: ingresos del mes	Coinciden con la suma del precio de las citas atendidas	☐
Veterinario: desactivar un servicio	Ya no se puede elegir al pedir una cita	☐
Veterinario: eliminar un servicio con citas	No se elimina; se muestra un mensaje	☐
Cerrar y abrir la app	Las citas siguen ahí y el perfil y el modo oscuro se recuerdan	☐

Crea el README.md con: nombre, descripción, árbol de paquetes, el diagrama MVVM, 6 capturas (3 de cada perfil) y las respuestas de 'Para pensar'.

COMMIT 15 **docs: agrega README con estructura y capturas**
Fin de la Fase 1.

```
git add .
git commit -m "docs: agrega README con estructura y capturas"
git push origin main
```

Resumen de commits de la Fase 1

N°	Mensaje del commit	Contenido
1	chore: importa proyecto base HuellitasVet	Proyecto base
2	feat(data): agrega entidades Usuario, Mascota y Cita	entity/
3	feat(data): agrega DAOs y relaciones CitaConDetalle y MascotaConDueno	dao/, relation/

N°	Mensaje del commit	Contenido
4	feat(data): registra entidades y siembra datos iniciales en VetDatabase	VetDatabase, DatosIniciales, VetApp
5	feat(data): implementa VetRepositoryRoom con todas las operaciones	repository/
6	feat(data): agrega PreferenciasStore con DataStore	prefs/
7	feat(viewmodel): agrega SesionViewModel y login con selector de rol	SesionViewModel, LoginScreen
8	feat(ui): agrega inicio del dueno y lista de mascotas	InicioViewModel + 2 pantallas
9	feat(ui): agrega formulario de mascota y ficha con historial clinico	MascotaViewModel + 2 pantallas
10	feat(ui): agrega pedir cita con horarios disponibles	PedirCitaViewModel + pantalla
11	feat(ui): agrega mis citas con cancelacion y regla de 2 horas	MisCitasViewModel + pantalla
12	feat(veterinario): agrega panel con graficos y agenda del dia	2 ViewModels + 2 pantallas
13	feat(veterinario): agrega atencion de citas y pacientes	2 ViewModels + 2 pantallas
14	feat: agrega servicios, perfil, ajustes y cambio de perfil	Formulario, Perfil, Ajustes, NavVet
15	docs: agrega README con estructura y capturas	README.md

FASE 2 · CON IA · Rama mejora-ia

Mejora profesional asistida por Gemini

Cuando la Fase 1 esté completa y subida a main, crea una rama nueva y usa Gemini en Android Studio para mejorar HuellitasVet sin romper la arquitectura MVVM + Repository.

```
git checkout -b mejora-ia
```

Mejoras entre las que eliges (mínimo 3)

Mejora	Qué debe lograr	Capas que cambian
Gráfico de peso de la mascota	La ficha muestra una línea con el peso registrado en cada atención (reutiliza GraficoLineas del laboratorio de Canvas)	DAO + Repository + ViewModel + View
Reprogramar cita	El dueño cambia fecha y hora de una cita PENDIENTE o CONFIRMADA respetando horarios libres y la regla de 2 horas	DAO + Repository + ViewModel + View
Carnet de vacunas	Nueva entidad Vacuna (mascotaId, nombre, fecha, próxima dosis) visible en la ficha	Entidad + DAO + Repository + ViewModel + View
Búsqueda de pacientes	Buscador por nombre de mascota o dueño y filtro por especie al mismo tiempo	ViewModel + View
Recordatorio de próximo control	Al atender una cita, el veterinario puede fijar una fecha de control y el dueño la ve en su Inicio	Entidad + DAO + ViewModel + View
Calendario mensual	Vista de mes en la agenda con puntos en los días que tienen citas	DAO + ViewModel + View

Condición obligatoria

Todo lo que genere la IA debe respetar MVVM: la View no usa DAO ni Repository; el ViewModel solo conoce la interface VetRepository; si agregas una función al Repository, la declaras primero en la interface. Si cambias o agregas una entidad, sube la version de la base. Si Gemini rompe estas reglas, lo corriges y lo anotas en PROMPTS.md.

Ejemplo de prompt (puedes mejorarlo)

```
Tengo una app Kotlin con Jetpack Compose, Material 3, MVVM, Room y un
VetRepository (interface). Agrega en la ficha de la mascota un grafico
de linea con el peso registrado en cada cita ATENDIDA. La consulta va
en CitaDao, la funcion en VetRepository y su implementacion, y la lista
de puntos se expone como StateFlow en MascotaViewModel. La View solo
dibuja el grafico con Canvas. No uses XML ni librerias externas.
Muestrame solo los archivos que cambian.
```

Commits mínimos de la Fase 2

COMMIT A	**ia: agrega grafico de peso en la ficha de la mascota**
COMMIT B	**ia: agrega reprogramacion de citas**
COMMIT C	**ia: agrega busqueda de pacientes**
COMMIT D	**docs: agrega PROMPTS.md** Documento con todos los prompts usados.

```
git push -u origin mejora-ia
```

Contenido de PROMPTS.md

```
# Prompts utilizados

## Prompt 1
**Texto enviado a Gemini:** ...
**Que genero la IA:** ...
**Que tuve que corregir yo:** ...
**Respeta MVVM?:** Si / No (explica)

## Reflexion (5 lineas)
- Que mejoro la IA y que no?
- Que error cometio?
- Que aprendi sobre MVVM, Room y reglas de negocio?
```